

Revisiting Classical and Deep Learning Methods for Image Classification

HaoTian Yuan
University of Manchester
14270372

haotian.yuan-4@postgrad.manchester.ac.uk

1. State-Of-The-Art for Robotics

1.1. Broad Overview

Early robotic vision relied on explicit programming and carefully engineered perception pipelines for well-specified tasks in controlled, static environments. During the past decade, advances in deep learning have substantially reshaped this landscape: by automatically learning latent patterns from data, deep models reduce the burden of manual feature design and enable robots to learn policies directly from high-dimensional sensory inputs [14]. In particular, deep imitation learning has made it possible to map observations to actions for complex behaviours in dynamic settings. However, imitation learning typically depends on large-scale labelled demonstrations, which are expensive to collect and computationally intensive to train [21].

To alleviate these constraints, *transfer learning* emerged as a practical solution: fine-tuning pre-trained visual backbones can significantly reduce the need for extensive real-world data collection while improving generalisation across tasks [34]. In parallel, *reinforcement learning* offers a complementary paradigm that does not require labelled action data, allowing robots to acquire behaviours through trial-and-error interaction [11]. Nevertheless, both paradigms face persistent challenges, including sample efficiency, safety during exploration, and the sim-to-real gap [31].

Beginning around 2023, robotics has increasingly leveraged large-scale foundation models—including Large Language Models (LLMs), Vision-Language Models (VLMs), and Vision-Language-Action (VLA) models—to support more flexible perception, instruction following, and decision-making [7, 35]. These models are often combined with preference alignment techniques (e.g., RLHF) for scenario-specific adaptation [24]. Despite this progress, robots still struggle to internalise real-world physical laws and to align learned representations with their embodied constraints [16].

This has motivated growing interest in *cognitive robotics* at the intersection of robotics, psychology, and biology [2]. Crucially, vision is a foundational modality for cognitive

robots: it provides the primary channel for building structured understanding of the environment, grounding symbols and language in perception, and maintaining situational awareness under uncertainty.

1.2. Environment Perception

Environment understanding forms the perceptual foundation of robotic autonomy, enabling robots to parse visual scenes into semantically meaningful representations that support downstream tasks such as navigation and manipulation. This capability has evolved substantially from early hand-crafted modular pipelines to modern deep learning method capable of real-time, end-to-end scene interpretation.

From Hand-Crafted Features to Deep Learning Traditional approaches to robotic vision relied on local feature descriptors (SIFT [19], SURF [1], and ORB [29]), combined with shallow classifiers or geometric reasoning. While computationally efficient and interpretable, these methods struggled with visual variability—changes in lighting, viewpoint, occlusion, and object appearance often led to brittle performance in unstructured environments. The emergence of deep convolutional neural networks (CNNs) fundamentally changed this landscape. Beginning with AlexNet’s breakthrough on ImageNet [13], and subsequently refined through architectures such as VGG [30] and ResNet [8], CNNs demonstrated that hierarchical feature learning could vastly outperform hand-crafted alternatives. ResNet, in particular, introduced residual connections that enabled training of substantially deeper networks, achieving robust feature extraction across diverse visual domains.

Object Detection and Instance Segmentation For robots operating in cluttered environments, identifying and localising individual objects is essential. Two-stage detectors such as Faster R-CNN [27] provide high accuracy by first proposing candidate regions and then classifying each region, making them suitable for manipulation tasks requiring precise localisation. Single-stage detectors such

as YOLO [26] and SSD [15] prioritise speed, enabling real-time detection critical for mobile robots and dynamic environments. Mask R-CNN [9] extended these capabilities to instance segmentation, providing pixel-level object masks that support fine-grained reasoning about object boundaries and affordances.

Semantic and Scene Understanding Beyond object-level detection, robots require holistic scene understanding to reason about traversability, spatial layout, and contextual relationships. Fully convolutional networks (FCN) [18] pioneered dense pixel-wise prediction for semantic segmentation, later refined by encoder-decoder architectures such as U-Net [28] and DeepLab [4] variants employing atrous convolutions and multi-scale processing. For robotic navigation, semantic segmentation enables distinguishing drivable surfaces from obstacles, while scene parsing supports higher-level reasoning about room types, functional zones, and spatial affordances.

More recently, Graph Neural Networks (GNNs) [33] have emerged as powerful tools for modelling inter-object relationships by representing detected entities as nodes within a graph structure. This relational reasoning is particularly advantageous for manipulation in cluttered scenes, where understanding spatial relationships (e.g., "the cup is on the table, next to the book") informs feasible grasp sequences and collision avoidance strategies.

Vision Transformers and Self-Supervised Learning The introduction of Vision Transformers (ViT) [6] marked a paradigm shift from convolution-based to attention-based visual processing. By treating images as sequences of patches and applying self-attention mechanisms, ViT captures long-range dependencies that CNNs struggle to model efficiently. Subsequent architectures such as Swin Transformer [17] introduced hierarchical structures and local attention windows, improving computational efficiency while maintaining global reasoning capabilities. DeiT [32] demonstrated that transformers can be trained effectively on smaller datasets through knowledge distillation, broadening their applicability to robotics scenarios with limited labelled data.

Self-supervised learning has further advanced robotic vision by enabling representation learning without extensive manual annotation. Methods such as DINO [3] and DINOv2 [23] learn powerful visual features through self-distillation, producing representations that transfer effectively to downstream tasks including segmentation, depth estimation, and object recognition. For robotics, where labelled data is scarce and expensive to collect, self-supervised pre-training offers a compelling path toward data-efficient perception systems.

Foundation Models for Robotic Perception The emergence of vision-language foundation models has opened new possibilities for robotic scene understanding. CLIP [25], trained on web-scale image-text pairs, enables zero-shot recognition by matching visual features to natural language descriptions—allowing robots to recognise novel objects specified only through language without task-specific fine-tuning. The Segment Anything Model (SAM) [10] provides class-agnostic segmentation capabilities, enabling robots to segment arbitrary objects from minimal prompts (points, boxes, or text), substantially reducing the annotation burden for new deployment environments.

Domain-specific foundation models have also emerged for robotics. R3M [22] and VC-1 [20] provide visual representations pre-trained on large-scale robot manipulation datasets, learning features that capture manipulation-relevant information (e.g., object affordances, gripper-object relationships) more effectively than ImageNet-pretrained backbones. These representations transfer efficiently to downstream manipulation tasks with minimal fine-tuning, addressing the data efficiency challenge inherent to robotic learning.

2. Dataset Description

CIFAR-100 [12] consists of 60,000 colour images of size 32×32 across 100 fine-grained categories (50,000 training and 10,000 test). Its low resolution and large class diversity make it a challenging benchmark that requires models to learn discriminative features from limited visual detail.

STL-10 [5] contains 10 natural image classes at a higher resolution of 96×96 , including 5,000 labelled training images, 8,000 test images, and 100,000 unlabelled samples for representation learning. The larger objects and clearer boundaries provide richer spatial structure for feature extraction.

Why these datasets?

The two datasets serve distinct but complementary purposes in our evaluation. CIFAR-100 is used primarily to assess the performance scaling of modern deep networks such as ResNet and ViT, as its fine-grained categories and low resolution expose the strengths and limitations of different architectural choices. In contrast, STL-10 provides higher-resolution images with fewer classes, making it a more suitable testbed for comparing classical methods against deep learning, since handcrafted features benefit from clearer object structure while deep models still maintain an advantage.

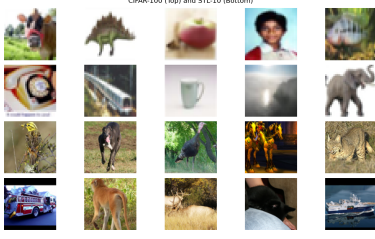


Figure 1. First Two Rows(cifar100); The Rest Two Rows(STL10)

3. Problem Definition

The task in this coursework is supervised image classification. Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, where each image x_i and its label y_i belong to one of C categories, the goal is to learn a function

$$f_{\theta}(x) \rightarrow y$$

that generalises to unseen samples. We consider two datasets with different characteristics: CIFAR-100 with 100 classes, and STL-10 with 10 classes.

To study how different modelling choices affect performance, we compare three approaches. The first is a Bag-of-Visual-Words pipeline, which extracts local descriptors, maps them to a visual vocabulary, and trains a shallow classifier. The second is a convolutional neural network such as ResNet, which learns features and a classifier jointly in an end-to-end manner. The third is a Vision Transformer model, trained either from scratch or through transfer learning. All methods are evaluated using the same preprocessing steps and the same train-test split, and we report Top-1 classification accuracy for comparison.

4. Deep Learning Methods

4.1. ResNet for Image Classification

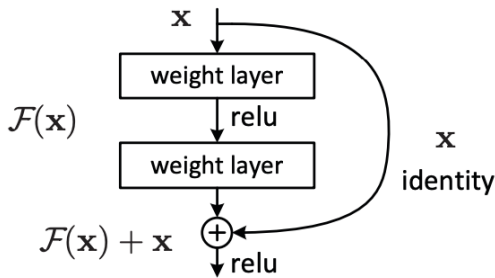


Figure 2. Residual Connection [8]

Residual Neural Networks (ResNets) are a class of deep convolutional architectures designed to address the optimisation challenges that emerge as network depth increases. Traditional CNNs suffer from vanishing gradients and the

degradation effect, where simply stacking more layers leads to worse training performance. ResNet mitigates this by introducing *residual connections*, allowing each block to learn a residual refinement rather than a full transformation.

A residual block learns a function $F(x)$ such that the overall mapping becomes

$$f(x) = F(x) + x,$$

where the skip connection enables more direct gradient flow to earlier layers. This simple architectural change makes it possible to train substantially deeper networks with stable convergence.

ResNet is structured as stacked convolution–batch-normalisation–ReLU layers grouped into stages of increasing feature dimensionality. Spatial resolution is reduced via strided convolutions, while semantic richness increases with depth. A global average pooling layer and fully connected classifier produce the final class predictions.

4.2. Vision Transformer for Image Classification

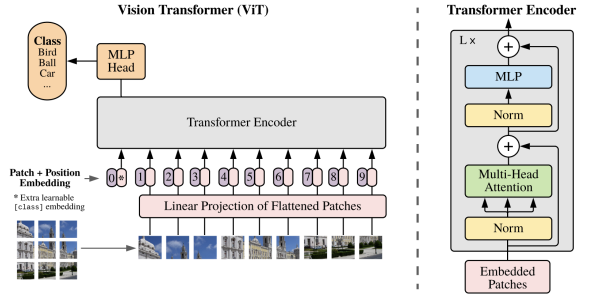


Figure 3. Vision Transformer [6]

The Vision Transformer (ViT) adapts the Transformer encoder to image classification by converting an image into a sequence of fixed-size patches. Each patch is linearly projected into an embedding vector, and positional information is added so that the model can distinguish patch locations. A learnable classification token is prepended to the sequence and is used to produce the final prediction.

The sequence is processed through multiple Transformer encoder layers, each composed of multi-head self-attention and a feed-forward network, both wrapped with residual connections. Through self-attention, ViT captures global relationships between all patches from the very beginning, in contrast to convolutional networks that build global context gradually.

The final representation of the classification token is passed to a linear prediction head to obtain the class label. Because ViT lacks the locality and translation biases inherent to CNNs, it typically requires large-scale pretraining to

reach strong performance. When such pretraining is available, ViT becomes a powerful alternative to convolution-based models, particularly in settings where global structure and long-range dependencies are important.

4.3. Experient Setting

4.3.1. Hyperparameter Search for ResNet Models

Table 1. Hyperparameters tested values for ResNet Models

Hyperparameter Name	Configuration
Architecture	ResNet18, ResNet34, ResNet50
Batch Size	16, 32, 64
Optimizer	{SGD, AdamW}
Initial Learning Rate	0.1, 0.01, 0.001
Weight Decay	1×10^{-4} , 1×10^{-2}

Table 1 summarises the hyperparameter search space used for tuning ResNet models on the CIFAR-100 dataset. We evaluate three architectures of increasing depth (ResNet18, ResNet34, and ResNet50) to analyse how network capacity affects classification performance.

We vary the **batch size** in {16, 32, 64} to study its influence on convergence behaviour and computational efficiency. Two commonly used **optimisers**, SGD and AdamW, are included to compare momentum-based optimisation with decoupled weight decay.

The **initial learning rate** is swept across three magnitudes (0.1, 0.01, 0.001), reflecting typical ranges for training convolutional networks from scratch. Finally, we explore a **weight decay** interval between 1×10^{-4} and 1×10^{-2} to evaluate how regularisation strength influences generalisation.

4.3.2. Hyperparameter Search for ViT Models

Table 2. Search space of hyperparameters for ViT on CIFAR-100.

Hyperparameter Name	Configuration
Model	ViT-B/16
Learning Rate	$1 \times 10^{-4} \sim 5 \times 10^{-4}$
Optimizer	SGD, AdamW
Batch Size	16, 32, 64
Weight Decay	$1 \times 10^{-4} \sim 1 \times 10^{-2}$

The hyperparameter search space for the ViT experiments is summarised in Table 2. We evaluate a ViT-B/16 backbone under a range of commonly used training configurations, including different learning rates, optimizers, batch sizes, and weight decay values. The ranges were selected to cover typical settings used when fine-tuning Transformers on medium-sized datasets such as CIFAR-100, while re-

maintaining computationally feasible. This setup allows us to analyse the sensitivity of ViT performance to training dynamics and regularisation strength.

4.4. Results Analysis

Architecture	Dataset	Param No.	Top-1 Acc(%)
ResNet18	CIFAR-100	11.22M	68.3 ± 0.18
ResNet34	CIFAR-100	21.33M	69.53 ± 0.12
ResNet50	CIFAR-100	23.71M	70.27 ± 0.10
ResNet101	CIFAR-100	42.70M	71.32 ± 0.15

Table 3. **Performance of ResNet architectures on CIFAR-100.** Reported results include mean Top-1 accuracy $\pm$ standard deviation over repeated runs.

As shown in Table 3, model performance increases steadily with network depth, with **ResNet101** achieving the highest Top-1 accuracy of **71.32%**. The parallel coordinate plot (Figure 4) supports this trend: higher-performing configurations consistently align with moderate learning rates (~ 0.03 – 0.06), larger batch sizes (64), and relatively stronger weight decay. These patterns suggest that deeper architectures benefit from more stable optimisation and heavier regularisation, whereas smaller models such as ResNet18 and ResNet34 exhibit greater sensitivity to hyperparameter choices, leading to both lower accuracy and higher variance across runs.

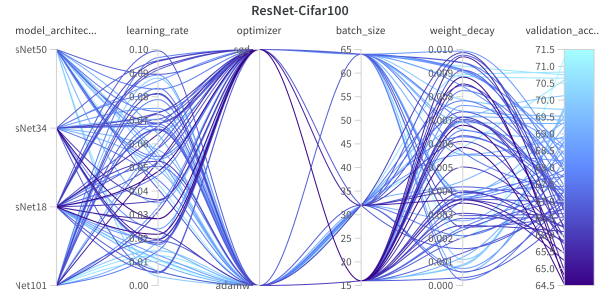


Figure 4. parallel coordinate diagram for Resnet

As shown in Figure 5, the overall performance of ViT-B/16 on CIFAR-100 exhibits a clear dependence on optimisation stability. The top-performing configuration reaches a Top-1 accuracy of **89.6%**, achieved under low learning rates ($\sim 1 \times 10^{-4}$ – 3×10^{-4}), moderate batch sizes (32–64), and stronger weight decay. The parallel coordinate plot further indicates that runs using AdamW consistently outperform SGD, with high-accuracy trajectories forming a tight cluster around well-regularised settings. In contrast, configurations with larger learning rates or insufficient regularisation diverge significantly, reflecting ViT’s sensitivity to training hyperparameters when trained on smaller datasets.

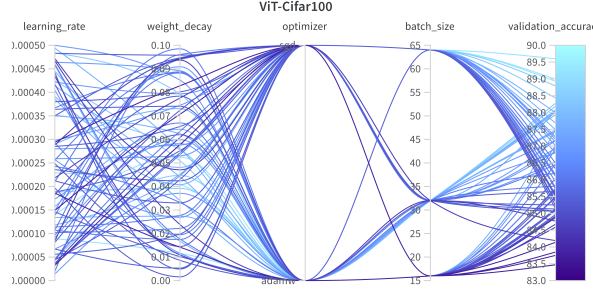


Figure 5. Parallel Coordinate Diagram for ViT

5. Bag-of-Visual-Words for Image Classification

The Bag-of-Visual-Words (BoVW) approach offers a classical pipeline for image classification, relying on handcrafted features rather than end-to-end learning. It consists of several sequential stages: feature extraction, visual vocabulary construction, feature encoding, and classification.

Feature Extraction The process begins by detecting keypoints in each image and computing descriptors around them. Methods such as SIFT, SURF, and ORB are widely used. SIFT captures distinctive gradient-based patterns, SURF provides a faster approximation with similar behaviour, and ORB offers a computationally efficient binary descriptor.

Codebook Construction Descriptors gathered from the training set are then grouped into clusters to create a visual vocabulary. Each cluster centre represents a recurring local pattern, often referred to as a “visual word”.

Feature Encoding To represent an image, each of its descriptors is matched to the nearest visual word, and the distribution of assignments is counted. This results in a histogram that captures how frequently different local patterns appear.

Classification The final stage involves training a classifier on these histogram features. Linear SVM, k-NN are commonly used choices, selected based on the desired trade-off between efficiency and predictive power.

5.1. Experiment Setting

As shown in Table 4, our BoVW pipeline explores the full classical vision workflow by varying feature extractors (SIFT, SURF, ORB), clustering algorithms (K-means and Gaussian Mixture Models), and classifiers (SVM and KNN). This configuration enables a systematic evaluation

Table 4. Experiment Setup and Hyperparameters

Component	Configuration
Feature Extraction	SIFT, SURF, ORB
Cluster Algorithm	Kmeans, Gaussian Mixture
Classifier	SVM, KNN

of how different handcrafted descriptors and unsupervised codebook constructions influence downstream classification performance, providing a meaningful contrast to the end-to-end learned representations used in deep neural networks.

5.2. Results Analysis

Dataset	Feature Extraction	Cluster	Classifier	Top-1 Acc (%)
cifar100	SIFT	K-means	SVM	9.72
STL10	SIFT	K-means	SVM	23.86

Table 5. best performance on two dataset

As shown in Table 5, the BoVW pipeline struggles on CIFAR-100, achieving only **9.72%** Top-1 accuracy. This poor performance is expected: CIFAR-100 contains 100 fine-grained categories at a very low resolution (32×32), making it difficult for handcrafted descriptors such as SIFT to extract stable and discriminative keypoints.

In contrast, the performance improves substantially on STL-10, reaching **23.86%**. The higher resolution and larger object scale in STL-10 provide richer texture and more reliable keypoints, allowing BoVW to form a more meaningful visual vocabulary. Nevertheless, the accuracy remains far below that of deep neural networks, highlighting the limitations of classical pipelines in complex recognition tasks and reinforcing the importance of learned hierarchical representations.

6. Comparative Analysis of BoVW, ResNet, and ViT

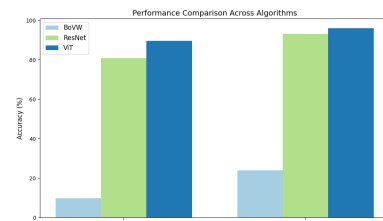


Figure 6. Performance Comparison on STL10

The bar chart 6 and the t-SNE visualizations [?] reveal clear differences in how the three methods learn visual

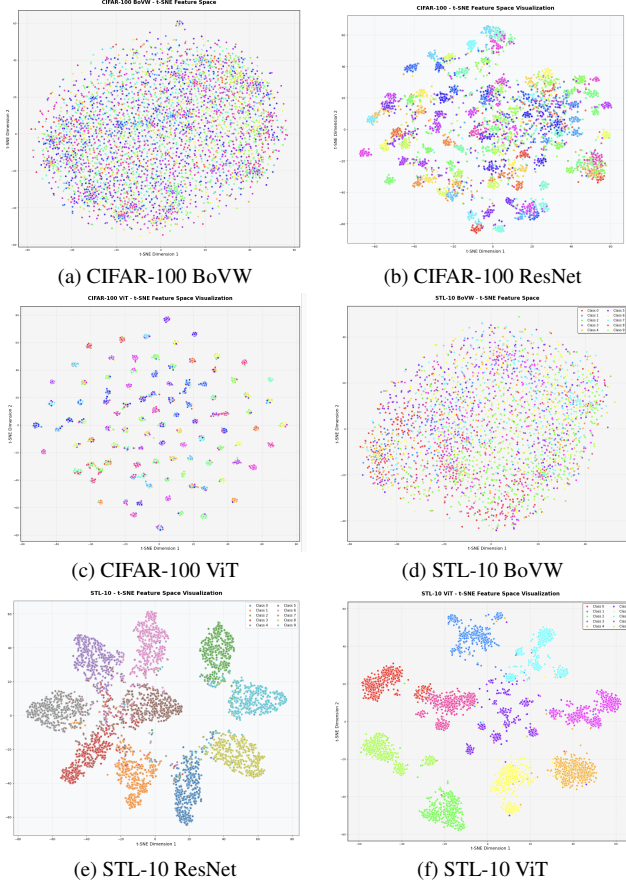


Figure 7. t-SNE visualizations of feature embeddings extracted by BoVW, ResNet, and ViT on CIFAR-100 and STL-10.

representations. BoVW produces diffuse, overlapping clusters on both datasets, reflecting its reliance on fixed, hand-crafted descriptors that cannot capture class-specific structure, particularly on low-resolution images such as CIFAR-100. ResNet yields far more coherent and well-separated clusters, consistent with its hierarchical feature extraction and strong inductive biases that support stable learning even under challenging visual conditions. ViT shows compact clusters on STL-10, where higher resolution allows attention to model global context effectively, but its features become scattered on CIFAR-100 due to the lack of locality priors and limited spatial detail. Overall, these patterns confirm that deep models outperform classical pipelines primarily through superior representation learning, while their effectiveness depends strongly on the visual properties of the dataset.

When to Use Classical vs. Deep Learning Methods. Classical approaches like BoVW remain useful when data are scarce, computation is limited, or interpretability and fast deployment are priorities. Their lightweight nature

makes them suitable for embedded or low-power robotic platforms. In contrast, deep learning methods should be preferred when large datasets are available, when robustness to variation is essential, or when the task demands rich semantic understanding. CNNs are ideal for low- and mid-resolution perception tasks, while ViT and other attention-based models excel in high-resolution environments or when pretrained models can be leveraged.

7. Conclusion

In this coursework, we first presented a structured overview of state-of-the-art deep learning methodologies for robotic vision, highlighting their evolution, capabilities, and limitations within modern robotic systems. Building on this foundation, we conducted a series of empirical studies on CIFAR-100 and STL-10 to compare classical computer vision pipelines with contemporary deep neural networks.

References

- [1] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. Surf: Speeded up robust features. In *European conference on computer vision*, pages 404–417. Springer, 2006. 1
- [2] Angelo Cangelosi and Minoru Asada. *Cognitive robotics*. MIT Press, 2022. 1
- [3] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021. 2
- [4] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation. *arXiv preprint arXiv:1706.05587*, 2017. 2
- [5] Adam Coates, Andrew Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 215–223. JMLR Workshop and Conference Proceedings, 2011. 2
- [6] Alexey Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 2, 3
- [7] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Ayaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, et al. Palm-e: An embodied multimodal language model. 2023. 1
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 1, 3
- [9] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017. 2
- [10] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer White-

- head, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4015–4026, 2023. 2
- [11] Jens Kober, J Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013. 1
- [12] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009. 2
- [13] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012. 1
- [14] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016. 1
- [15] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C Berg. Ssd: Single shot multibox detector. In *European conference on computer vision*, pages 21–37. Springer, 2016. 2
- [16] Yang Liu, Weixing Chen, Yongjie Bai, Xiaodan Liang, Guanbin Li, Wen Gao, and Liang Lin. Aligning cyber space with physical world: A comprehensive survey on embodied ai. *IEEE/ASME Transactions on Mechatronics*, 2025. 1
- [17] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021. 2
- [18] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015. 2
- [19] David G Lowe. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60(2):91–110, 2004. 1
- [20] Arjun Majumdar, Karmesh Yadav, Sergio Arnaud, Jason Ma, Claire Chen, Sneha Silwal, Aryan Jain, Vincent-Pierre Berges, Tingfan Wu, Jay Vakil, et al. Where are we in the search for an artificial visual cortex for embodied intelligence? *Advances in Neural Information Processing Systems*, 36:655–677, 2023. 2
- [21] Ajay Mandlekar, Danfei Xu, Josiah Wong, Soroush Nasiriany, Chen Wang, Rohun Kulkarni, Li Fei-Fei, Silvio Savarese, Yuke Zhu, and Roberto Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. *arXiv preprint arXiv:2108.03298*, 2021. 1
- [22] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. *arXiv preprint arXiv:2203.12601*, 2022. 2
- [23] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023. 2
- [24] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022. 1
- [25] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021. 2
- [26] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016. 2
- [27] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, 28, 2015. 1
- [28] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015. 2
- [29] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary Bradski. Orb: An efficient alternative to sift or surf. In *2011 International conference on computer vision*, pages 2564–2571. Ieee, 2011. 1
- [30] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014. 1
- [31] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017. 1
- [32] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021. 2
- [33] Xiao Zang, Miao Yin, Jinqi Xiao, Saman Zonouz, and Bo Yuan. Graphmp: Graph neural network-based motion planning with efficient graph search. *Advances in Neural Information Processing Systems*, 36:3131–3142, 2023. 2
- [34] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1):43–76, 2020. 1
- [35] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023. 1

Appendix A: ResNet

```
1 import torch
2 import torch.nn as nn
3
4 class BasicBlock(nn.Module):
5     expansion = 1
6     def __init__(self, in_channels,
7                   out_channels, stride=1):
8         super().__init__()
9         self.residual = nn.Sequential(
10             nn.Conv2d(in_channels,
11                       out_channels, 3, stride, 1,
12                       bias=False),
13             nn.BatchNorm2d(out_channels),
14             nn.ReLU(inplace=True),
15             nn.Conv2d(out_channels,
16                       out_channels, 3, 1, 1, bias=
17                       False),
18             nn.BatchNorm2d(out_channels)
19         )
20
21     def forward(self, x):
22         return nn.ReLU(inplace=True)(self.
23             residual(x) + self.shortcut(x))
24
25 class Bottleneck(nn.Module):
26     expansion = 4
27     def __init__(self, in_channels,
28                   out_channels, stride=1):
29         super().__init__()
30         self.residual = nn.Sequential(
31             nn.Conv2d(in_channels,
32                       out_channels, 1, bias=False)
33             ,
34             nn.BatchNorm2d(out_channels),
35             nn.ReLU(inplace=True),
36             nn.Conv2d(out_channels,
37                       out_channels, 3, stride, 1,
38                       bias=False),
39             nn.BatchNorm2d(out_channels),
40             nn.ReLU(inplace=True),
41             nn.Conv2d(out_channels,
42                       out_channels * 4, 1, bias=
43                       False),
44             nn.BatchNorm2d(out_channels *
45                             4)
46         )
```

```
39 self.shortcut = nn.Sequential()
40 if stride != 1 or in_channels !=
41     out_channels * 4:
42     self.shortcut = nn.Sequential(
43         nn.Conv2d(in_channels,
44                   out_channels * 4, 1,
45                   stride, bias=False),
46         nn.BatchNorm2d(out_channels
47                         * 4)
48     )
49
50 def forward(self, x):
51     return nn.ReLU(inplace=True)(self.
52         residual(x) + self.shortcut(x))
53
54 class ResNet(nn.Module):
55     def __init__(self, block, layers,
56                 num_classes=100):
57         super().__init__()
58         self.in_channels = 64
59         self.conv1 = nn.Sequential(
60             nn.Conv2d(3, 64, 3, 1, 1, bias=
61                 False),
62             nn.BatchNorm2d(64),
63             nn.ReLU(inplace=True)
64         )
65         self.conv2 = self._make_layer(block
66             , 64, layers[0], 1)
67         self.conv3 = self._make_layer(block
68             , 128, layers[1], 2)
69         self.conv4 = self._make_layer(block
70             , 256, layers[2], 2)
71         self.conv5 = self._make_layer(block
72             , 512, layers[3], 2)
73         self.pool = nn.AdaptiveAvgPool2d
74             ((1, 1))
75         self.fc = nn.Linear(512 * block.
76             expansion, num_classes)
77
78     def _make_layer(self, block,
79                     out_channels, blocks, stride):
80         strides = [stride] + [1] * (blocks
81             - 1)
82         layers = []
83         for st in strides:
84             layers.append(block(self.
85                 in_channels, out_channels,
86                 st))
87             self.in_channels = out_channels
88                 * block.expansion
89         return nn.Sequential(*layers)
90
91     def forward(self, x):
92         x = self.conv1(x)
93         x = self.conv2(x)
94         x = self.conv3(x)
95         x = self.conv4(x)
96         x = self.conv5(x)
```

```

79         x = self.pool(x)
80         x = x.view(x.size(0), -1)
81         return self.fc(x)
82
83     def resnet18(num_classes=100):
84         return ResNet(BasicBlock, [2,2,2,2],
85                        num_classes)
86
87     def resnet34(num_classes=100):
88         return ResNet(BasicBlock, [3,4,6,3],
89                        num_classes)
90
91     def resnet50(num_classes=100):
92         return ResNet(Bottleneck, [3,4,6,3],
93                        num_classes)
94
95     import argparse, os, time
96     import torch
97     import torch.nn as nn
98     import torch.optim as optim
99     from torch.utils.tensorboard import
100         SummaryWriter
101     import torchvision.transforms as transforms
102     import torchvision.datasets as datasets
103     import wandb
104     from model import resnet18, resnet34,
105         resnet50
106
107     parser = argparse.ArgumentParser()
108     parser.add_argument('-net', type=str,
109                        required=True)
110     parser.add_argument('-gpu', action='
111         store_true')
112     parser.add_argument('-b', type=int, default
113         =128)
114     parser.add_argument('-lr', type=float,
115         default=0.1)
116     parser.add_argument('-wd', type=float,
117         default=5e-4)
118     parser.add_argument('-epochs', type=int,
119         default=200)
120     parser.add_argument('-wandb', action='
121         store_true')
122     args = parser.parse_args()
123
124     transform_train = transforms.Compose([
125         transforms.RandomCrop(32, padding=4),
126         transforms.RandomHorizontalFlip(),
127         transforms.ToTensor()
128     ])
129
130     transform_test = transforms.Compose([
131         transforms.ToTensor()])
132
133     train_set = datasets.CIFAR100(root='./data'
134                                   , train=True, download=True, transform=
135                                   transform_train)
136
137     test_set = datasets.CIFAR100(root='./data'
138                                   , train=False, download=True, transform=
139                                   transform_test)
140
141     train_loader = torch.utils.data.DataLoader(
142         train_set, batch_size=args.b, shuffle=
143         True, num_workers=4)
144     test_loader = torch.utils.data.DataLoader(
145         test_set, batch_size=args.b, shuffle=
146         False, num_workers=4)
147
148     nets = {
149         "resnet18": resnet18,
150         "resnet34": resnet34,
151         "resnet50": resnet50,
152     }
153
154     net = nets[args.net](num_classes=100)
155     device = torch.device("cuda" if args.gpu
156                           else "cpu")
157     net = net.to(device)
158
159     criterion = nn.CrossEntropyLoss()
160     optimizer = optim.SGD(net.parameters(), lr=
161         args.lr, momentum=0.9, weight_decay=args
162         .wd)
163     scheduler = optim.lr_scheduler.MultiStepLR(
164         optimizer, milestones=[60,120,160],
165         gamma=0.2)
166
167     writer = SummaryWriter()
168
169     if args.wandb:
170         wandb.init(project="resnet-cifar100",
171                    config=vars(args))
172
173     best_acc = 0
174
175     for epoch in range(args.epochs):
176         net.train()
177         for x, y in train_loader:
178             x, y = x.to(device), y.to(device)
179             optimizer.zero_grad()
180             loss = criterion(net(x), y)
181             loss.backward()
182             optimizer.step()
183
184         net.eval()
185         correct = 0
186         total = 0
187         with torch.no_grad():
188             for x, y in test_loader:
189                 x, y = x.to(device), y.to(
190                     device)
191                 outputs = net(x)
192                 _, pred = outputs.max(1)
193                 correct += pred.eq(y).sum().
194                     item()

```

```

165         total += y.size(0)
166
167     acc = correct / total
168     scheduler.step()
169
170     writer.add_scalar("acc", acc, epoch)
171     if args.wandb:
172         wandb.log({"acc": acc})
173
174     if acc > best_acc:
175         best_acc = acc
176         torch.save(net.state_dict(), f"{
177             args.net}_best.pth")
178
179 writer.close()
180 if args.wandb:
181     wandb.finish()
182
183 import argparse, torch
184 import torchvision.transforms as transforms
185 from model import resnet18, resnet34,
186     resnet50
187
188 parser = argparse.ArgumentParser()
189 parser.add_argument('-net', type=str,
190     required=True)
191 parser.add_argument('-weights', type=str,
192     required=True)
193 parser.add_argument('-gpu', action='
194     store_true')
195 parser.add_argument('-b', type=int, default
196     =64)
197 args = parser.parse_args()
198
199 transform = transforms.Compose([transforms.
200     ToTensor()])
201 test_set = datasets.CIFAR100(root='./data',
202     train=False, download=True, transform=
203     transform)
204 test_loader = torch.utils.data.DataLoader(
205     test_set, batch_size=args.b, shuffle=
206     False, num_workers=4)
207
208 nets = {
209     "resnet18": resnet18,
210     "resnet34": resnet34,
211     "resnet50": resnet50,
212 }
213
214 net = nets[args.net](num_classes=100)
215 net.load_state_dict(torch.load(args.weights
216 ))
217 device = torch.device("cuda" if args.gpu
218     else "cpu")
219 net = net.to(device)
220 net.eval()

```

```

210 correct1 = 0
211 correct5 = 0
212 total = 0
213
214 with torch.no_grad():
215     for x, y in test_loader:
216         x, y = x.to(device), y.to(device)
217         outputs = net(x)
218         _, pred = outputs.topk(5, 1, True,
219             True)
220         correct = pred.eq(y.view(-1,1).
221             expand_as(pred))
222         correct1 += correct[:, :1].sum().
223             item()
224         correct5 += correct[:, :5].sum().
225             item()
226         total += y.size(0)
227
228 print("Top-1_Acc:", correct1 / total)
229 print("Top-5_Acc:", correct5 / total)

```

Appendix B: ViT

```

1 import torch
2 from torch.utils.data import DataLoader
3 import torchvision
4 import torchvision.transforms as transforms
5
6 imagenet_mean = [0.485, 0.456, 0.406]
7 imagenet_std = [0.229, 0.224, 0.225]
8
9 def get_cifar100_loaders(root, batch_size,
10     workers):
11     train_transform = transforms.Compose([
12         transforms.Resize(224),
13         transforms.RandomHorizontalFlip(),
14         transforms.RandomCrop(224, padding
15             =4),
16         transforms.RandAugment(num_ops=2,
17             magnitude=9),
18         transforms.ToTensor(),
19         transforms.Normalize(mean=
20             imagenet_mean, std=imagenet_std)
21     ])
22
23     test_transform = transforms.Compose([
24         transforms.Resize(224),
25         transforms.ToTensor(),
26         transforms.Normalize(mean=
27             imagenet_mean, std=imagenet_std)
28     ])
29
30     train_set = torchvision.datasets.
31         CIFAR100(root=root, train=True,
32         download=False, transform=
33         train_transform)

```

```

26     test_set = torchvision.datasets.
        CIFAR100(root=root, train=False,
        download=False, transform=
        test_transform)
27
28     train_loader = DataLoader(train_set,
        batch_size=batch_size, shuffle=True,
        num_workers=workers, pin_memory=
        True, persistent_workers=True)
29     test_loader = DataLoader(test_set,
        batch_size=batch_size, shuffle=False
        , num_workers=workers, pin_memory=
        True, persistent_workers=True)
30
31     return train_loader, test_loader
32
33 def get_stl10_loaders(root, batch_size,
34     workers):
35     train_transform = transforms.Compose([
36         transforms.Resize(224),
37         transforms.RandomHorizontalFlip(),
38         transforms.RandomCrop(224, padding
39             =4),
40         transforms.RandAugment(num_ops=2,
41             magnitude=9),
42         transforms.ToTensor(),
43         transforms.Normalize(mean=
44             imagenet_mean, std=imagenet_std)
45     ])
46
47     test_transform = transforms.Compose([
48         transforms.Resize(224),
49         transforms.ToTensor(),
50         transforms.Normalize(mean=
51             imagenet_mean, std=imagenet_std)
52     ])
53
54     train_set = torchvision.datasets.STL10(
55         root=root, split='train', download=
56         False, transform=train_transform)
57     test_set = torchvision.datasets.STL10(
58         root=root, split='test', download=
59         False, transform=test_transform)
60
61     train_loader = DataLoader(train_set,
62         batch_size=batch_size, shuffle=True,
63         num_workers=workers, pin_memory=
64         True, persistent_workers=True)
65     test_loader = DataLoader(test_set,
66         batch_size=batch_size, shuffle=False
67         , num_workers=workers, pin_memory=
68         True, persistent_workers=True)
69
70     return train_loader, test_loader
71
72 import torch
73 import torch.nn as nn
74
75 import torch.optim as optim
76 from torch.cuda.amp import GradScaler,
77     autocast
78 from model_vit import create_vit
79 from data import get_cifar100_loaders,
80     get_stl10_loaders
81 import wandb
82 import argparse
83
84 def train_epoch(model, dataloader,
85     criterion, optimizer, device, scaler):
86     model.train()
87     total_loss = 0
88     total_correct = 0
89     total = 0
90     for x, y in dataloader:
91         x = x.to(device, non_blocking=True)
92         y = y.to(device, non_blocking=True)
93         optimizer.zero_grad()
94         with autocast():
95             out = model(x)
96             loss = criterion(out, y)
97             scaler.scale(loss).backward()
98             scaler.step(optimizer)
99             scaler.update()
100         total_loss += loss.item() * x.size
101         (0)
102         _, pred = out.max(1)
103         total_correct += pred.eq(y).sum().
104         item()
105         total += y.size(0)
106     return total_loss / total, (
107         total_correct / total) * 100
108
109 def evaluate(model, dataloader, criterion,
110     device):
111     model.eval()
112     total_loss = 0
113     total_correct = 0
114     total = 0
115     with torch.no_grad():
116         for x, y in dataloader:
117             x = x.to(device, non_blocking=
118                 True)
119             y = y.to(device, non_blocking=
120                 True)
121             out = model(x)
122             loss = criterion(out, y)
123             total_loss += loss.item() * x.
124             size(0)
125             _, pred = out.max(1)
126             total_correct += pred.eq(y).sum
127             ().item()
128             total += y.size(0)
129     return total_loss / total, (
130         total_correct / total) * 100

```

```

105 def train_pipeline(dataset, data_root,
106                    epochs, lr, wd, batch_size, workers,
107                    use_wandb):
108     device = torch.device("cuda" if torch.
109                           cuda.is_available() else "cpu")
110
111     if dataset == "cifar100":
112         train_loader, test_loader =
113             get_cifar100_loaders(data_root,
114                                 batch_size, workers)
115         model = create_vit(num_classes=100,
116                           pretrained=True)
117         save_name = "vit_cifar100_best.pth"
118     else:
119         train_loader, test_loader =
120             get_stl10_loaders(data_root,
121                              batch_size, workers)
122         model = create_vit(num_classes=10,
123                           pretrained=True)
124         save_name = "vit_stl10_best.pth"
125
126     model = model.to(device)
127
128     if use_wandb:
129         wandb.init(
130             project="vit-training",
131             config={
132                 "dataset": dataset,
133                 "epochs": epochs,
134                 "lr": lr,
135                 "weight_decay": wd,
136                 "batch_size": batch_size,
137                 "workers": workers,
138                 "optimizer": "AdamW",
139                 "scheduler": "
140                     CosineAnnealingLR"
141             }
142         )
143
144     criterion = nn.CrossEntropyLoss(
145         label_smoothing=0.1)
146     optimizer = optim.AdamW(model.
147                             parameters(), lr=lr, weight_decay=wd
148                             )
149     scheduler = optim.lr_scheduler.
150         CosineAnnealingLR(optimizer, T_max=
151         epochs, eta_min=1e-6)
152     scaler = GradScaler()
153
154     best_acc = 0
155
156     for epoch in range(epochs):
157         train_loss, train_acc = train_epoch
158             (model, train_loader, criterion,
159             optimizer, device, scaler)
160         test_loss, test_acc = evaluate(
161             model, test_loader, criterion,
162             device)
163
164         scheduler.step()
165
166         if use_wandb:
167             wandb.log({
168                 "train_loss": train_loss,
169                 "train_acc": train_acc,
170                 "test_loss": test_loss,
171                 "test_acc": test_acc,
172                 "lr": scheduler.get_last_lr
173                     () [0]
174             })
175
176         if test_acc > best_acc:
177             best_acc = test_acc
178             torch.save(model.state_dict(),
179                       save_name)
180             if use_wandb:
181                 artifact = wandb.Artifact(f
182                     "{dataset}_vit_best",
183                     type="model")
184                 artifact.add_file(save_name
185                                 )
186                 wandb.log_artifact(artifact
187                                 )
188
189         if use_wandb:
190             wandb.finish()
191
192 if __name__ == "__main__":
193     parser = argparse.ArgumentParser()
194     parser.add_argument("--dataset", type=
195         str, choices=["cifar100", "stl10"],
196         required=True)
197     parser.add_argument("--data_root", type
198         =str, default="./data")
199     parser.add_argument("--epochs", type=
200         int, default=30)
201     parser.add_argument("--lr", type=float,
202         default=3e-4)
203     parser.add_argument("--wd", type=float,
204         default=0.05)
205     parser.add_argument("--batch_size",
206         type=int, default=128)
207     parser.add_argument("--workers", type=
208         int, default=8)
209     parser.add_argument("--wandb", type=
210         bool, default=False)
211     args = parser.parse_args()
212
213     train_pipeline(
214         dataset=args.dataset,
215         data_root=args.data_root,
216         epochs=args.epochs,
217         lr=args.lr,
218         wd=args.wd,
219         batch_size=args.batch_size,
220         workers=args.workers,
221         use_wandb=args.wandb

```

```

187     )
188
189 import torch
190 import torch.nn as nn
191 import numpy as np
192
193 def extract_vit_features(model, dataloader,
194     device, max_samples=5000):
195     model.eval()
196     features_list = []
197     labels_list = []
198     with torch.no_grad():
199         for x, y in dataloader:
200             x = x.to(device, non_blocking=
201                 True)
202             b = x.size(0)
203             x_proj = model.conv_proj(x)
204             x_proj = x_proj.flatten(2).
205                 transpose(1, 2)
206             cls = model.class_token.expand(
207                 b, -1, -1)
208             x_proj = torch.cat([cls, x_proj
209                 ], dim=1)
210             x_encoded = model.encoder(
211                 x_proj)
212             cls_token = x_encoded[:, 0]
213             features_list.append(cls_token.
214                 cpu().numpy())
215             labels_list.append(y.numpy())
216             if sum(len(a) for a in
217                 labels_list) >= max_samples:
218                 break
219     features = np.concatenate(features_list
220         , axis=0)[:max_samples]
221     labels = np.concatenate(labels_list,
222         axis=0)[:max_samples]
223     return features, labels
224
225 import numpy as np
226 from sklearn.manifold import TSNE
227
228 def compute_tsne(features, random_seed=42,
229     perplexity=30, max_iter=1000):
230     tsne = TSNE(
231         n_components=2,
232         perplexity=perplexity,
233         random_state=random_seed,
234         max_iter=max_iter,
235         verbose=0
236     )
237     out = tsne.fit_transform(features)
238     return np.array(out)
239
240 import matplotlib.pyplot as plt
241 import numpy as np
242
243 def plot_tsne(points, labels, num_classes,
244     title, colors=None, figsize=(8, 8)):

```

```

233     if colors is None:
234         colors = plt.cm.tab20(np.linspace
235             (0, 1, num_classes))
236
237     fig = plt.figure(figsize=figsize)
238     ax = fig.add_subplot(1, 1, 1)
239
240     for c in range(num_classes):
241         mask = labels == c
242         if mask.sum() > 0:
243             ax.scatter(
244                 points[mask, 0],
245                 points[mask, 1],
246                 c=[colors[c]],
247                 s=15,
248                 alpha=0.8,
249                 edgecolors="none"
250             )
251
252     ax.set_title(title, fontsize=16)
253     ax.set_xticks([])
254     ax.set_yticks([])
255     plt.tight_layout()
256     return fig

```

Appendix C: BoVW

```

1 import cv2
2 import numpy as np
3 import torch
4
5 def tensor_to_cv2_image(tensor):
6     img = tensor.permute(1, 2, 0).cpu().
7         numpy()
8     img = (img * 255).astype(np.uint8)
9     return cv2.cvtColor(img, cv2.
10         COLOR_RGB2GRAY)
11
12 class LocalFeatureExtractor:
13     def __init__(self, method="SIFT",
14         max_keypoints=200):
15         self.method = method.upper()
16         self.max_keypoints = max_keypoints
17         self.extractor = self.
18             _create_extractor()
19
20     def _create_extractor(self):
21         if self.method == "SIFT":
22             return cv2.SIFT_create()
23         if self.method == "ORB":
24             return cv2.ORB_create(nfeatures
25                 =self.max_keypoints)
26         if self.method == "SURF":
27             try:
28                 return cv2.xfeatures2d.
29                     SURF_create(

```

```

25         hessianThreshold=400)
26     except:
27         return None
28     return None
29
30 def _truncate(self, keypoints,
31 descriptors):
32     if descriptors is None or len(
33 descriptors) == 0:
34         return np.zeros((1, 128), dtype
35 =np.float32)
36
37     if len(descriptors) <= self.
38 max_keypoints:
39         return descriptors.astype(np.
40 float32)
41
42     responses = np.array([kp.response
43 for kp in keypoints])
44 top_idx = np.argsort(responses)[-
45 self.max_keypoints:]
46 return descriptors[top_idx].astype(
47 np.float32)
48
49 def extract_single(self, tensor_img):
50 img = tensor_to_cv2_image(
51 tensor_img)
52 keypoints, descriptors = self.
53 extractor.detectAndCompute(img,
54 None)
55 return self._truncate(keypoints,
56 descriptors)
57
58 def extract_dataset(self, dataset,
59 max_samples=None):
60 descriptors_list = []
61 labels = []
62 n = len(dataset) if max_samples is
63 None else min(max_samples, len(
64 dataset))
65 for i in range(n):
66     x, y = dataset[i]
67     desc = self.extract_single(x)
68     descriptors_list.append(desc)
69     labels.append(y)
70 return descriptors_list, np.array(
71 labels)
72
73 import numpy as np
74 from sklearn.cluster import MiniBatchKMeans
75 import pickle
76 import wandb
77
78 class VisualVocabulary:
79     def __init__(self, n_clusters=500,
80 batch_size=2000, use_wandb=False):
81         self.n_clusters = n_clusters
82         self.batch_size = batch_size
83
84         self.use_wandb = use_wandb
85         self.kmeans = MiniBatchKMeans(
86             n_clusters=n_clusters,
87             batch_size=batch_size,
88             random_state=42,
89             max_iter=100,
90             verbose=0
91         )
92
93     def _merge_descriptors(self,
94 descriptor_list):
95         arrays = [d for d in
96 descriptor_list if d is not None
97 and len(d) > 0]
98         if len(arrays) == 0:
99             return np.zeros((1, 128), dtype
100 =np.float32)
101         return np.vstack(arrays)
102
103     def fit(self, descriptor_list):
104         all_desc = self._merge_descriptors(
105 descriptor_list)
106         self.kmeans.fit(all_desc)
107
108         if self.use_wandb:
109             wandb.log({
110                 "vocab_cluster_count": self
111                     .n_clusters,
112                 "vocab_train_descriptor_count": len(all_desc),
113                 "vocab_inertia": self.
114                     kmeans.inertia_
115             })
116
117     def predict(self, descriptors):
118         return self.kmeans.predict(
119 descriptors)
120
121     def save(self, path):
122         with open(path, "wb") as f:
123             pickle.dump(self.kmeans, f)
124
125     def load(self, path):
126         with open(path, "rb") as f:
127             self.kmeans = pickle.load(f)
128
129 import numpy as np
130 import wandb
131
132 class BoVWEncoder:
133     def __init__(self, vocab, use_wandb=
134 False):
135         self.vocab = vocab
136         self.use_wandb = use_wandb
137         self.n_clusters = vocab.n_clusters
138
139     def encode_single(self, descriptors):

```

```

112         if descriptors is None or len(
113             descriptors) == 0:
114             hist = np.zeros(self.n_clusters
115                             , dtype=np.float32)
116             return hist
117
118         words = self.vocab.predict(
119             descriptors)
120         hist, _ = np.histogram(words, bins=
121             self.n_clusters, range=(0, self.
122             n_clusters))
123         hist = hist.astype(np.float32)
124
125         norm = np.linalg.norm(hist)
126         if norm > 1e-6:
127             hist /= norm
128
129         return hist
130
131     def encode_dataset(self,
132         descriptor_list, labels=None):
133         histograms = []
134         for desc in descriptor_list:
135             histograms.append(self.
136                 encode_single(desc))
137
138         histograms = np.array(histograms,
139             dtype=np.float32)
140
141         if self.use_wandb:
142             wandb.log({
143                 "bovw_hist_mean": np.mean(
144                     histograms),
145                 "bovw_hist_std": np.std(
146                     histograms),
147                 "bovw_hist_nonzero_ratio":
148                     np.mean(histograms > 0)
149             })
150
151         return histograms
152
153 import numpy as np
154 from sklearn.svm import SVC
155 from sklearn.neighbors import
156     KNeighborsClassifier
157 from sklearn.preprocessing import
158     StandardScaler
159 from sklearn.metrics import accuracy_score
160 import wandb
161
162 class BoVWClassifier:
163     def __init__(self, method="svm", k=3,
164         use_wandb=False):
165         self.method = method.lower()
166         self.use_wandb = use_wandb
167         self.scaler = None
168
169         if self.method == "svm":
170             self.model = SVC(kernel="linear
171                             ", C=1.0, max_iter=2000)
172         elif self.method == "knn":
173             self.model =
174                 KNeighborsClassifier(
175                     n_neighbors=k)
176         else:
177             raise ValueError("Unsupported_
178                 classifier._Use_'svm'_or_'
179                 knn'._")
180
181     def fit(self, X, y):
182         if self.method == "svm":
183             self.scaler = StandardScaler()
184             X = self.scaler.fit_transform(X
185                 )
186             self.model.fit(X, y)
187
188     def predict(self, X):
189         if self.method == "svm":
190             X = self.scaler.transform(X)
191             return self.model.predict(X)
192
193     def evaluate(self, X_train, y_train,
194         X_test, y_test):
195         train_pred = self.predict(X_train)
196         test_pred = self.predict(X_test)
197
198         train_acc = accuracy_score(y_train,
199             train_pred) * 100
200         test_acc = accuracy_score(y_test,
201             test_pred) * 100
202
203         if self.use_wandb:
204             wandb.log({
205                 "train_accuracy": train_acc
206                 ,
207                 "test_accuracy": test_acc,
208                 "classifier": self.method
209             })
210
211         return train_acc, test_acc
212
213 import numpy as np
214 from sklearn.manifold import TSNE
215
216 def tsne_embed(
217     features,
218     max_samples=5000,
219     perplexity=30,
220     random_state=42
221 ):
222     n = len(features)
223     if n > max_samples:
224         idx = np.random.choice(n,
225             max_samples, replace=False)

```

```

203     selected_features = features[idx]
204     return TSNE(
205         n_components=2,
206         perplexity=perplexity,
207         random_state=random_state,
208         init="pca",
209         max_iter=1000
210     ).fit_transform(selected_features),
211         idx
212 else:
213     embeddings = TSSE(
214         n_components=2,
215         perplexity=perplexity,
216         random_state=random_state,
217         init="pca",
218         max_iter=1000
219     ).fit_transform(features)
220     idx = np.arange(n)
221     return embeddings, idx
222
223 import numpy as np
224 import matplotlib.pyplot as plt
225 from matplotlib.colors import
226     ListedColormap
227
228 def visualize_tsne(points, labels,
229     num_classes, title):
230     if num_classes <= 10:
231         colors = np.array([
232             [1.0, 0.0, 0.0],
233             [0.0, 0.5, 1.0],
234             [0.0, 1.0, 0.0],
235             [1.0, 0.0, 1.0],
236             [1.0, 0.65, 0.0],
237             [0.5, 0.0, 1.0],
238             [1.0, 1.0, 0.0],
239             [0.0, 1.0, 1.0],
240             [1.0, 0.08, 0.58],
241             [0.5, 1.0, 0.0]
242         ])
243     elif num_classes <= 20:
244         colors = plt.cm.tab20(np.linspace
245             (0, 1, 20))
246     else:
247         colors = plt.cm.hsv(np.linspace(0,
248             0.95, num_classes))
249
250 fig = plt.figure(figsize=(24, 10))
251 ax1 = plt.subplot(1, 2, 1)
252 for i in range(num_classes):
253     m = labels == i
254     if m.sum() > 0:
255         ax1.scatter(points[m, 0],
256             points[m, 1], c=[colors[i]],
257             alpha=0.8, s=50, edgecolors
258                 ='white', linewidth=1.0)
259 ax1.set_title(f'{title}_Border')
260 ax1.set_facecolor('#f0f0f0')

```

```

253 ax2 = plt.subplot(1, 2, 2)
254 ax2.scatter(points[:, 0], points[:, 1],
255     c=labels, cmap=ListedColormap(
256         colors), alpha=0.85, s=30)
257 ax2.set_title(f'{title}_Density')
258 ax2.set_facecolor('#f0f0f0')
259 plt.tight_layout()
260 plt.show()
261
262 fig = plt.figure(figsize=(16, 14))
263 ax = plt.gca()
264 for i in range(num_classes):
265     m = labels == i
266     if m.sum() > 0:
267         ax.scatter(points[m, 0], points
268             [m, 1], c=[colors[i]], alpha
269                 =0.8, s=60, edgecolors='
270                 white', linewidth=1.2)
271 ax.set_title(f'{title}_High-Res')
272 ax.set_facecolor('#f5f5f5')
273 plt.tight_layout()
274 plt.show()
275
276 fig = plt.figure(figsize=(14, 12))
277 ax = plt.gca()
278 fig.patch.set_facecolor('black')
279 ax.set_facecolor('black')
280 for i in range(num_classes):
281     m = labels == i
282     if m.sum() > 0:
283         ax.scatter(points[m, 0], points
284             [m, 1], c=[colors[i]], alpha
285                 =0.7, s=40)
286 ax.set_title(f'{title}_Art', color='
287     white')
288 ax.tick_params(colors='white')
289 plt.tight_layout()
290 plt.show()

```